Assignment 07 Knowledge Document

Classes and Objects with Inheritance and Version Control

Student Name: Stephen Kabati

Date: August 11, 2026

Course: Introduction to Programming with Python

Instructor: RRoot

Assignment: Assignment 07 – Classes and Objects

GitHub Repository URL: https://github.com/Stephen-Kabati/Assignment07

Table of Contents

1. Introduction

This assignment demonstrates the application of Object-Oriented Programming (OOP) principles in Python, specifically focusing on classes, objects, constructors, properties, inheritance, and method overriding. Building upon the foundational concepts from Module 06 (functions and separation of concerns), Module 07 introduces data organization through custom classes rather than simple dictionary structures.

The program developed for this assignment is a Course Registration Program that manages student enrollment data using a Student data class inherited from a Person parent class. The program reads from and writes to a JSON file (Enrollments.json), demonstrating the complete data lifecycle from persistent storage to object-oriented manipulation in memory.

2. Development Process

Step 1: Review of Module Materials

Before writing any code, I systematically reviewed all attached module materials to ensure a thorough understanding of the concepts:

Mod07-Notes.pdf was studied for theoretical foundations of classes versus objects, data encapsulation, constructors, properties, abstraction, encapsulation, and inheritance. Demo01-ClassesAndFunctions.py was reviewed to understand how classes organize functions and how objects maintain independent data states (data encapsulation). Demo02-UsingConstructors.py demonstrated the use of __init__ constructors to initialize object attributes. Demo03-UsingProperties.py showed how to implement getter and setter properties with the @property decorator for controlled attribute access and validation. Demo04-UsingInheritence.py illustrated creating a Person parent class and a Student child class using super().__init__() to inherit and extend functionality. Demo05-ConvertingJsonAndStudentObjects.py explained the critical pattern of converting between JSON dictionaries and custom Python objects for file I/O operations. DblAndSingleUnderscores.py reviewed Python's naming conventions for private attributes. Mod07-Lab01-WorkingWithConstructors.py, Mod07-Lab02-WorkingWithProperties.py, and Mod07-Lab03-WorkingWithInheritance.py provided step-by-step patterns for each major concept. Assignment07-Starter.py was used as the base structure, and Enrollments.json served as the starter data file.

Step 2: Designing the Person Class

Following the pattern established in Mod07-Lab03-WorkingWithInheritance.py and Demo04-UsingInheritence.py, I created a Person class that implicitly inherits from Python's base object class. The class includes: a constructor (__init__) that initializes first_name and last_name; private attributes (__first_name, __last_name) using double underscores for encapsulation, as discussed in Mod07-Notes (page 14) and demonstrated in DblAndSingleUnderscores.py; property getters that return title-cased names for consistent formatting; property setters that validate input using isalpha() to prevent numeric characters, raising ValueError when validation fails; and an overridden __str__() method that returns a comma-separated string representation.

Step 3: Designing the Student Class (Inheritance)

The Student class was designed to inherit from Person, as required by the assignment and demonstrated in Mod07-Lab03. I used class Student(Person) to establish the parent-child relationship, called super().__init__(first_name=first_name, last_name=last_name) to leverage the parent constructor, eliminating redundant code. I added a course_name property specific to students and overrode the __str__() method again to include course_name in the comma-separated output, demonstrating method overriding as described in Mod07-Notes (page 23).

Step 4: Updating FileProcessor Class

Referencing Demo05-ConvertingJsonAndStudentObjects.py and Mod07-Lab01-WorkingWithConstructors.py, I modified read_data_from_file() to read JSON dictionary rows and convert each dictionary into a Student object before appending to the list. I modified write_data_to_file() to convert Student objects back into dictionaries before using json.dump(), since the JSON module cannot serialize custom objects directly.

Step 5: Updating IO Class

Following the Assignment07-Starter.py structure and Mod07-Lab02 property validation patterns, I updated input_student_data() to create Student objects instead of dictionaries. I maintained structured error handling for first name and last name input validation. I updated output_student_and_course_names() to print each Student object directly,

relying on the overridden __str__() method to produce comma-separated values as required by the assignment.

Step 6: Error Handling

I implemented structured error handling as specified in the assignment requirements. For file reading, try-except blocks in read_data_from_file() catch FileNotFoundError and general exceptions. For user input, try-except blocks in input_student_data() catch ValueError from both manual checks and property setters. For file writing, try-except blocks in write_data_to_file() catch TypeError and general exceptions.

3. Code Architecture and Key Concepts

Separation of Concerns

The program follows the three-layer architecture emphasized throughout the course. The Data Layer consists of the Person and Student data classes that manage data structure, state, and validation. The Processing Layer is the FileProcessor class that handles all file I/O operations and data conversion. The Presentation Layer is the IO class that manages all user interaction, input collection, and output formatting. This separation is a direct application of the Separation of Concerns pattern discussed in Module 06 and reinforced in Mod07-Notes (page 3).

Data Encapsulation

Each object maintains its own memory space and data, as illustrated in Mod07-Notes (page 4) with the cookie-cutter analogy. The use of private attributes (__first_name, __last_name, __course_name) ensures that data can only be accessed through controlled property methods, preventing accidental modification.

Properties as Accessors and Mutators

Properties provide a Pythonic way to implement getters and setters without changing the external syntax of attribute access. As demonstrated in Demo03-UsingProperties.py and Mod07-Lab02, the @property decorator creates the getter, while @name.setter creates

the setter. This allows validation logic to run transparently whenever an attribute is modified.

JSON Conversion Pattern

Since Python's json module only works with native types (dict, list, str, int, float, bool), custom objects must be converted to dictionaries before saving and converted back after loading. This pattern, central to Demo05, involves two steps. Loading: json.load() produces a list of dicts, then each dict is iterated to instantiate Student objects. Saving: Student objects are iterated to build a list of dicts, then json.dump() is used to write to file.

4. Answers to Focus Questions

4.1 What are the differences between statements, functions, and classes?

Statements are individual instructions that perform actions, such as assignments (x = 5), loops (while True), conditionals (if x > 0), or output (print(x)). They are the smallest executable units of a program.

Functions are named blocks of statements designed to perform a specific task. Functions promote code reuse by allowing the same block of code to be called multiple times with different arguments. As shown in Mod07-Notes (page 2), functions wrap statements into reusable units.

Classes are blueprints or templates that organize both data (attributes and properties) and behavior (methods and functions) into a single structure. Classes serve as factories for creating objects. As noted in Mod07-Notes (page 3), functions are organized into classes when a programmer believes they will be used many times in many programs. Classes enable Object-Oriented Programming principles like encapsulation, inheritance, and polymorphism.

4.2 What is the difference between a data class, a presentation class, and a processing class?

Data Classes focus primarily on storing and managing data. They typically contain attributes, constructors, and properties with validation logic. In this assignment, Person and Student are data classes because their primary responsibility is to hold and validate student information (Mod07-Notes, page 5).

Presentation Classes focus on user interaction, displaying output and collecting input. In this assignment, the IO class is a presentation class because it contains methods like output_menu(), input_menu_choice(), and output_student_and_course_names() that manage all console interaction.

Processing Classes focus on performing actions and transformations on data, such as reading from or writing to files. In this assignment, the FileProcessor class is a processing class because it handles JSON serialization and deserialization without any user interaction (Mod07-Notes, page 5).

4.3 What is a constructor?

A constructor is a special method automatically called when an object is instantiated. In Python, the constructor is named __init__. Its primary purpose is to initialize the object's attributes to a defined starting state. As explained in Mod07-Notes (page 6), constructors are commonly used in object-oriented programming languages like Python, Java, and C++ to assign initial values to the object's member variables. In this assignment, both Person.__init__() and Student.__init__() set up the initial values for their respective attributes.

4.4 What is an attribute?

An attribute is a variable that holds data specific to an object. Attributes describe the state or characteristics of an object. For example, in the Student class, first_name, last_name, and course_name are attributes. As defined in Mod07-Notes (page 6), attributes are variables that hold data specific to an object and are used to store the state or characteristics of an object. Each instance of a class maintains its own independent set of attribute values.

4.5 What is a property?

A property is a special type of method that controls access to an attribute. Properties typically come in pairs: a getter (accessor) for reading data and a setter (mutator) for writing data. In Python, the @property decorator defines a getter, and @name.setter defines a setter. As demonstrated in Demo03-UsingProperties.py and Mod07-Lab02, properties enable data validation, formatting (like .title()), and abstraction while allowing the external code to use simple attribute syntax (obj.name = value) instead of method calls (obj.set_name(value)).

4.6 What is class inheritance?

Class inheritance is an OOP mechanism where a new class (child or subclass) derives attributes and methods from an existing class (parent or superclass). This promotes code reuse and establishes logical is-a relationships. In this assignment, Student(Person) indicates that a Student is a Person with additional attributes. As explained in Mod07-Notes (page 21), a new class can inherit data and behaviors from an existing class. The child class uses super().__init__() to invoke the parent's constructor.

4.7 What is an overridden method?

An overridden method is a method in a child class that replaces a method inherited from a parent class. The child method has the same name and parameters but provides customized behavior. In this assignment, both Person and Student override the __str__() method inherited from Python's base object class. As shown in Mod07-Notes (page 23) and Demo04-UsingInheritence.py, overriding allows each class to define its own string representation. Person returns first,last while Student returns first,last,course.

4.8 What is the difference between Git and GitHub Desktop?

Git is a command-line version control system (VCS) that tracks changes to files, manages branching and merging, and maintains a complete history of a project. It operates locally on a developer's machine (Mod07-Notes, page 31).

GitHub Desktop is a free graphical user interface (GUI) application that simplifies Git operations for users who prefer visual interaction over command-line syntax. It handles cloning, committing, branching, and pushing or pulling to remote GitHub repositories through point-and-click interactions. As described in Mod07-Notes (page 32) and Demo06-Using PyCharm Git Integration.txt, GitHub Desktop simplifies your development workflow by providing a visual layer on top of Git commands.

5. Testing and Validation

PyCharm Testing

I opened the project in PyCharm and ensured Enrollments.json was present in the working directory. I ran Assignment07.py and verified the menu displayed correctly. I selected option 1 and entered valid student data (first name: John, last name: Doe, course: Python 200). I verified the confirmation message appeared and the student object was created with property validation. I selected option 2 and confirmed comma-separated output: John,Doe,Python 200. I selected option 3 and verified Enrollments.json was updated with the new student as a dictionary row. I tested invalid input (numeric first name) and confirmed ValueError was caught and displayed gracefully.

Console and Terminal Testing

I navigated to the script directory in the OS command shell. I ran python Assignment07.py and verified identical behavior to PyCharm execution, confirming the program runs independently of the IDE. I confirmed that Enrollments.json was readable and writable from the console environment.

6. Conclusion

This assignment successfully demonstrates the transition from procedural programming (using dictionaries and standalone functions) to object-oriented programming (using custom classes with inheritance and properties). By implementing the Person and Student classes, the program now benefits from Data Encapsulation, where private attributes prevent unauthorized modification; Validation, where property setters ensure data

integrity at the point of entry; Inheritance, where shared code for first and last name handling is centralized in the parent class; Polymorphism, where overridden __str__() methods provide meaningful string representations; and Persistence, where JSON conversion patterns allow object data to survive between program executions.

These foundational OOP concepts, as systematically presented in Mod07-Notes and reinforced through Labs 01 through 03 and Demos 01 through 05, form the basis for building more complex, maintainable, and scalable software systems.

7. References

RRoot. (2030). Mod07-Notes: Classes and Objects [PDF]. Module 07 Course Materials.

RRoot. (2030). Assignment07-Starter.py [Python script]. Module 07 Assignment.

RRoot. (2030). Mod07-Lab01-WorkingWithConstructors.py [Python script]. Module 07 Lab 01.

RRoot. (2030). Mod07-Lab02-WorkingWithProperties.py [Python script]. Module 07 Lab 02.

RRoot. (2030). Mod07-Lab03-WorkingWithInheritance.py [Python script]. Module 07 Lab 03.

RRoot. (2030). Demo01-ClassesAndFunctions.py [Python script]. Module 07 Demo 01.

RRoot. (2030). Demo02-UsingConstructors.py [Python script]. Module 07 Demo 02.

RRoot. (2030). Demo03-UsingProperties.py [Python script]. Module 07 Demo 03.

RRoot. (2030). Demo04-UsingInheritence.py [Python script]. Module 07 Demo 04.

RRoot. (2030). Demo05-ConvertingJsonAndStudentObjects.py [Python script]. Module 07 Demo 05.

RRoot. (2030). DblAndSingleUnderscores.py [Python script]. Module 07 Supplemental.

RRoot. (2030). Mod07-Assignment.pdf [PDF]. Module 07 Assignment Instructions.